

Практическое занятие 1. Базовый бэкенд: модель, миграции, админ-панель, валидация

Цель: Научиться создавать Django-проект, определять модель данных с валидацией, выполнять миграции и работать с админ-панелью. На этом занятии мы не делаем собственный фронтенд, всё проверяем через встроенную админку Django.

Продолжительность: 2–3 академических часа.

Примечание: Все действия выполняются в операционной системе Ubuntu, которая установлена на корпоративных ноутбуках. Использование собственных компьютеров не допускается.

1. Подготовка среды

Откройте терминал (Ctrl + Alt + T). Создайте папку для проекта и перейдите в неё:

```
bash
mkdir training_exam
cd training_exam
```

Создайте виртуальное окружение и активируйте его:

```
bash
python3 -m venv venv
source venv/bin/activate
```

Установите Django:

```
bash
pip install django
```

2. Создание проекта и приложения

Создайте проект `exam_project`:

```
bash

django-admin startproject exam_project

cd exam_project
```

Создайте приложение `products` (для модели «Товары»):

```
bash

python manage.py startapp products
```

3. Регистрация приложения

Откройте файл `exam_project/settings.py`. Найдите список `INSTALLED_APPS` и добавьте в него `'products'`:

```
python

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'products',  # наше приложение
]
```

4. Создание модели с валидацией

В файле `products/models.py` опишем модель `Product`. Добавим валидацию на уровне модели (метод `clean`), чтобы проверять:

- `name` не пустое.

- `price` больше нуля.
- `sku` уникальный (это уже задано атрибутом `unique=True`, но дополнительную проверку можно добавить).

python

```
from django.db import models
from django.core.exceptions import ValidationError

class Product(models.Model):
    name = models.CharField(max_length=150, verbose_name="Название")
    category = models.CharField(max_length=100, verbose_name="Категория")
    price = models.DecimalField(max_digits=10, decimal_places=2,
verbose_name="Цена")
    sku = models.CharField(max_length=50, unique=True,
verbose_name="Артикул")
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Дата
создания")

    def __str__(self):
        return self.name

    def clean(self):
        """
        Кастомная валидация для модели.
        Вызывается автоматически при сохранении через форму (в том числе в
админке).
        """
        # Проверка, что название не пустое
        if not self.name or self.name.strip() == '':
            raise ValidationError({'name': 'Название товара не может быть
пустым.'})

        # Проверка, что цена больше нуля
        if self.price <= 0:
            raise ValidationError({'price': 'Цена должна быть больше
нуля.'})

        # Проверка уникальности sku (хотя unique=True делает это на уровне
БД,
        # но здесь мы можем сделать более красивую ошибку)
        if
Product.objects.filter(sku=self.sku).exclude(pk=self.pk).exists():
            raise ValidationError({'sku': 'Товар с таким артикулом уже
существует.'})
```

```
def save(self, *args, **kwargs):
    # Вызываем валидацию перед сохранением
    self.full_clean()

    super().save(*args, **kwargs)
```

Пояснения:

- `full_clean()` вызывает `clean()` и валидацию каждого поля.
 - Если валидация не пройдена, будет выброшено `ValidationError`, и сохранение не произойдёт.
 - В админке Django эти ошибки будут показаны пользователю в виде сообщений.
-

5. Создание и применение миграций

Выполните команды для создания миграций и обновления базы данных:

```
bash

python manage.py makemigrations

python manage.py migrate
```

6. Регистрация модели в админ-панели

Откройте файл `products/admin.py` и зарегистрируйте модель:

```
python

from django.contrib import admin
from .models import Product

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ('id', 'name', 'category', 'price', 'sku', 'created_at')
    list_filter = ('category',)

    search_fields = ('name', 'sku')
```

Это сделает админ-панель удобной для проверки.

7. Создание суперпользователя

Чтобы войти в админку, создайте суперпользователя:

```
bash

python manage.py createsuperuser
```

Следуйте инструкциям (логин, email, пароль).

8. Запуск сервера и проверка валидации

Запустите сервер:

```
bash

python manage.py runserver
```

Перейдите в браузере по адресу `http://127.0.0.1:8000/admin` и войдите под суперпользователем.

Проверка валидации:

1. Попробуйте добавить новый товар с пустым названием – админка должна показать ошибку.
2. Попробуйте добавить товар с ценой 0 или отрицательной – ошибка.
3. Попробуйте добавить два товара с одинаковым артикулом – ошибка.

Если валидация работает, значит вы всё сделали правильно.

9. Обработка ошибки 404

Django автоматически возвращает страницу 404 в следующих случаях:

- При попытке открыть в админке объект с несуществующим ID (например, `http://127.0.0.1:8000/admin/products/product/9999/`).
- Если в вашем коде используется функция `get_object_or_404` (мы будем её использовать на следующем занятии).

Для проверки просто перейдите по несуществующему ID в админке – вы увидите стандартную страницу «404 Not Found».

10. Задания для самостоятельной работы

Задание 1 (обязательное).

Создайте ещё одно приложение `events` в том же проекте. В нём реализуйте модель `Event` согласно второму тренировочному варианту (мероприятия).

Поля:

- `title` (`CharField`, `max_length=200`, не пустое)
- `location` (`CharField`, `max_length=150`)
- `event_date` (`DateTimeField`)
- `max_guests` (`IntegerField`)
- `created_at` (`DateTimeField`, `auto_now_add=True`)

Валидация:

- `title` не может быть пустым.
- `event_date` должна быть в будущем (можно использовать `date.today()` или `timezone.now()`).
- `max_guests` должно быть больше нуля.

Дополнительные указания:

- Зарегистрируйте модель в админ-панели.
- Выполните миграции.
- Проверьте валидацию через админку (попробуйте создать мероприятие с датой в прошлом).

Задание 2 (для закрепления).

Добавьте в модель `Product` поле `stock` (`IntegerField`, `default=0`) и валидацию: количество на складе не может быть отрицательным.

Задание 2 (дополнительное).

Создайте новое приложение `orders` в том же проекте. В нём реализуйте модель `Order` согласно описанию ниже.

Поля модели Order

Поле	Тип данных	Ограничения
id	INTEGER (PK)	автоинкремент
customer_name	CharField (max_length=200)	не пустое
product_name	CharField (max_length=200)	не пустое
quantity	PositiveIntegerField	> 0
order_date	DateField	по умолчанию сегодня, не может быть в будущем
status	CharField (max_length=50)	один из: 'new', 'processing', 'shipped', 'delivered'
created_at	DateTimeField	автоматически

Валидация

- `customer_name` – не может быть пустым.
- `product_name` – не может быть пустым.
- `quantity` – целое положительное число (>0).
- `order_date` – не может быть больше текущей даты (нельзя создать заказ в будущем).
- `status` – должен быть одним из допустимых значений (можно использовать `choices` в модели).

Что должно быть в итоге

- Работающий проект Django с двумя приложениями (`products` и `events`).
- Модель `Product` с валидацией (пустое имя, цена >0 , уникальный `sku`).
- Модель `Event` с валидацией (пустое название, дата в будущем, `max_guests` >0).
- Обе модели зарегистрированы в админ-панели.
- Админ-панель позволяет выполнять CRUD-операции и показывает ошибки валидации.
- Страница 404 при попытке редактировать несуществующий объект.